EGC

SPÉCIFICATIONS DE HAUT NIVEAU

Application mobile & web de l'Église Glorieuse du Christ

Description fonctionnelle et technique de la solution, de son périmètre actuel, de son administration et des prérequis avant mise en production.

Version du document **1.0**	Date **19 octobre 2026**
Plateformes **Web responsive · Android**	État **Prototype fonctionnel en évolution**

Ce document décrit le comportement observé dans le code du projet. Il ne constitue ni une certification de sécurité ni une garantie de disponibilité en production.

01 · VUE D'ENSEMBLE

Objectifs et périmètre

L'application EGC est un point d'accès numérique aux contenus, activités et démarches de l'Église Glorieuse du Christ. Elle associe une interface React accessible sur le Web et sous Android à une API Express et une base SQLite pour les inscriptions et l'administration.

Objectifs utilisateurs

- Découvrir les actualités, événements et contenus de la communauté.
- Accéder aux lectures bibliques, aux ressources et aux directs.
- Retrouver les assemblées et partager l'application avec ses proches.
- Envoyer une demande d'inscription ou de préinscription.

Objectifs administratifs

- Consulter les demandes reçues et leur statut.
- Confirmer ou refuser une inscription.
- Visualiser les intentions de don sans les confondre avec des paiements reçus.

Publics concernés

Fidèles, visiteurs, personnes intéressées par les formations, équipe administrative de l'église et équipe technique chargée du déploiement et de la maintenance.

Sommaire

1. Vue d'ensemble et périmètre
2. Architecture et technologies
3. Fonctionnalités de l'application
4. Administration et parcours métier
5. API et modèle de données
6. Sécurité et protection des données
7. Installation et exploitation
8. Limites actuelles et feuille de route

> **Point de lecture essentiel :** les statuts « disponible », « dépendant d'une configuration » et « à développer » sont distingués dans ce document pour ne pas présenter une maquette ou une intention comme une fonction opérationnelle.

02 · CONCEPTION TECHNIQUE

Architecture générale

Client EGC
React · Vite
Web responsive

⇄

Conteneur Android
Capacitor
Projet Gradle

⇄

API EGC
Node.js · Express
Port 8080

Interface client

Application monopage React, navigation par routes, styles adaptatifs aux téléphones et écrans larges, thème clair/sombre. Vite assemble les ressources Web.

Application Android

Capacitor encapsule la version Web dans un projet Android natif. Les ressources doivent être resynchronisées après les changements du front-end.

API et stockage

API REST Express avec validation des données et SQLite. La base locale est créée au premier démarrage à partir du schéma applicatif.

Services externes

Google Maps pour la recherche d'itinéraire, WhatsApp pour le partage, YouTube pour le direct si les identifiants de chaîne et de clé API sont configurés.

Socle technologique constaté

Couche	Technologie	Rôle
Front-end	React 19, React Router 7, Vite 7	Écrans, navigation et production des ressources Web.
Mobile	Capacitor 8, Android/Gradle	Emballage et exécution de l'interface sur Android.
Back-end	Node.js ≥ 22.13, Express 5	API HTTP, authentification administrateur et opérations métier.
Base de données	SQLite (API Node intégrée)	Administrateurs, inscriptions, demandes de don, clés d'idempotence.
Sécurité	bcrypt, JWT, Helmet, CORS, Zod, rate limit	Hachage, session, validation et protection des routes d'administration.

Architecture de prototype : SQLite est prévu pour une instance unique sur un disque persistant. Ce choix et l'adresse locale de développement ne constituent pas un hébergement de production.

03 · APPLICATION PUBLIQUE

Fonctionnalités côté utilisateur

Rubrique	Description	État actuel
Accueil	Tableau de bord, mise en avant des actualités, verset et accès aux rubriques. Un contenu d'annonce local est utilisé en secours.	Flux annonces à raccorder
Événements	Écran de consultation et filtres par catégories (culte, formation, prière, événements spéciaux, retraite).	Dépend de l'API annonces
Bible & lecture	Accès à la lecture biblique, plan de lecture, historique de lecture et rappels selon les écrans disponibles.	Écrans présents
Direct	Recherche périodique d'un direct YouTube; sinon lien vers la chaîne.	Clé et chaîne YouTube requises
Formations	Choix d'une formation, collecte nom/prénom/téléphone/e-mail et envoi d'une préinscription.	API connectée
Inscription	Formulaire d'inscription communautaire avec réponse affichant l'état de la demande.	API connectée
Églises	Trois fiches : Brazzaville (Dix Maisons, en face de la station), Pointe-Noire (à côté de l'hôtel Elais) et Oyo. Liens Google Maps de recherche et d'itinéraire.	Repères fournis
Invitation	Ouvre WhatsApp avec un message prêt à partager. Le lien de téléchargement peut être défini via `VITE_APP_DOWNLOAD_URL`.	Partage disponible
Dons et dîmes	La demande MTN peut enregistrer une intention non payée. Airtel, cartes et PayPal ne réalisent pas de paiement.	Aucun paiement actif
Prière	L'écran explique que l'envoi de demande de prière est en cours de développement.	À développer
Profil, encouragement, témoignages, À propos	Rubriques de navigation et contenus présents dans le front-end.	Contenu/API à confirmer

Localisation et invitations

Les adresses des églises sont utilisées comme termes de recherche Google Maps : aucun point GPS précis n'est affirmé. Les coordonnées exactes doivent être recueillies auprès des responsables avant de publier une géolocalisation garantie.

WhatsApp est ouvert sur le téléphone/navigateur de l'utilisateur; l'application ne lit pas le carnet d'adresses et n'envoie pas de message à sa place. La sélection du contact et l'envoi restent une action volontaire de l'utilisateur.

04 · GESTION INTERNE

Espace administrateur

L'espace d'administration se trouve sur la route `/admin`. L'accès est réservé à un compte administrateur initialisé par configuration locale; aucune inscription publique d'administrateur n'est prévue.

Fonctions présentes

- Connexion par e-mail et mot de passe; le jeton signé est conservé en mémoire dans l'interface et expire au bout de 20 minutes.
- Consultation des demandes d'inscription et filtrage par statut.
- Changement du statut d'une inscription en « confirmée » ou « refusée ».
- Consultation des demandes de don avec étiquette explicite indiquant que le prestataire de paiement n'est pas configuré.

Cycle de vie d'une inscription

1 · Demande

La personne soumet ses coordonnées et l'activité; le statut initial est « à examiner ».

2 · Examen

L'administrateur consulte la demande et décide de la confirmer ou de la refuser.

3 · Résultat

Une demande confirmée pour la même activité et le même téléphone ne peut pas être répétée.

Règle anti-doublon : l'activité et le téléphone normalisé forment une clé unique. Une demande en attente ou refusée peut être mise à jour et resoumise; une demande confirmée bloque une nouvelle inscription pour le même couple.

Cycle de vie d'un don

Le front-end envoie une demande avec une clé d'idempotence. Le back-end conserve une intention MTN au statut `pending_provider`. Ce statut signifie « non payée, prestataire non raccordé »; aucune confirmation ni preuve de paiement n'est créée par le système.

Ne pas comptabiliser ces demandes comme des recettes : l'écran d'administration peut les afficher, mais le back-end ne prélève aucun montant et ne possède pas encore de confirmation MTN.

Compte et responsabilités

L'e-mail et le secret initial sont configurés dans `server/.env` sur la machine d'hébergement. Les secrets ne doivent figurer ni dans le dépôt, ni dans le PDF, ni dans un message ou une capture d'écran.

05 · CONTRATS APPLICATIFS

API et modèle de données

Le serveur de développement écoute sur le port `8080`. Les routes métier implémentées dans l'API Express dédiée sont les suivantes :

Méthode & route	Accès	Usage
`GET /api/health`	Public	Contrôle élémentaire de disponibilité.
`POST /api/admin/auth/login`	Public limité	Authentification administrateur et émission d'un JWT.
`POST /api/inscription`	Public limité	Création ou mise à jour d'une demande d'inscription.
`POST /api/formations`	Public limité	Création ou mise à jour d'une demande de formation.
`POST /api/pay`	Public limité	Enregistrement idempotent d'une intention MTN non payée.
`GET /api/admin/registrations`	Administrateur	Liste et filtre des inscriptions.
`PATCH /api/admin/registrations/:id`	Administrateur	Modification du statut en confirmée ou refusée, sous contraintes métier.
`GET /api/admin/donations`	Administrateur	Consultation des intentions de don enregistrées.

Entités persistées

Table	Rôle et données principales	Contraintes notables
`admins`	Comptes administrateurs, rôle et mot de passe haché.	Adresse e-mail unique; aucune création publique.
`registrations`	Type, activité, identité, contact, réponses, statut et dates.	Unicité activité + téléphone normalisé.
`donation_intents`	Nom, téléphone, montant en XAF, opérateur, statut et référence prestataire.	Le statut ne représente pas une confirmation de paiement.
`idempotency_requests`	Clé, type et empreinte de requête, identifiant de ressource.	Clé unique; conflit si réutilisée pour un contenu différent.

Idempotence : une relance de la même requête de don avec la même clé renvoie la même intention. La même clé utilisée avec des données différentes est refusée. Les écritures importantes utilisent des transactions SQLite.

Dépendance d'intégration à signaler : le front-end interroge aussi `/api/annonces` pour les actualités/événements. Cette route n'est pas fournie par la nouvelle API Express décrite ici; un service d'annonces existant ou une intégration distincte est nécessaire.

06–07 · PROTECTION ET DÉPLOIEMENT

Sécurité, données et exploitation

Mesures implémentées

- Validation structurée des entrées côté serveur; requêtes SQL paramétrées et transactions pour protéger les opérations sensibles.
- Mots de passe hachés avec bcrypt; JWT d'administration à durée limitée; vérification d'accès sur les routes privées.
- Limitation des tentatives de connexion, des écritures publiques et des requêtes administrateur.
- En-têtes de sécurité Helmet, CORS configuré par liste d'origines, taille des requêtes JSON limitée.
- Clés d'idempotence et contraintes de base de données pour limiter les doublons.
- Fichiers locaux `.env` et base SQLite exclus du contrôle de version.

Informations personnelles manipulées

Les formulaires d'inscription peuvent conserver nom, prénom, téléphone, e-mail facultatif et réponses. Les intentions de don conservent nom, téléphone et montant proposé. Avant toute collecte réelle à grande échelle, publier une politique de confidentialité, limiter les accès, définir les durées de conservation et organiser les demandes de rectification/suppression conformément aux règles applicables.

Démarrage de développement

Action	Commande / configuration	Remarque
Démarrer l'interface Web	`npm run dev`	Vite, généralement accessible sur le port 3000 ou le port affiché.
Démarrer le backend	`npm run server:dev`	Depuis la racine du projet; écoute sur 8080 par défaut.
Configurer le backend	`server/.env`	Contient secret JWT, compte administrateur et chemin SQLite; ne pas publier.
Pointer le front-end	`VITE_API_URL`	URL de l'API. La valeur de développement par défaut est localhost:8080.
Synchroniser Android	`npm run android:sync`	Reconstruit le front-end puis copie les assets dans le projet natif.

Prérequis techniques : Node.js 22.13 ou plus récent pour l'API SQLite intégrée; Android Studio, SDK Android et JDK 21 pour la compilation Android.

Attention Android : un appareil ou émulateur ne peut pas utiliser `localhost` pour joindre le PC hôte. Configurer une URL réseau joignable; en production, utiliser HTTPS et restreindre CORS aux origines réelles.

08 · PRÉPARATION DE LA SUITE

Limites et priorités recommandées

Priorité	Action recommandée	Résultat attendu
P0	Déployer le front-end et l'API sur des adresses HTTPS publiques et configurées; préparer un stockage persistant.	Accès cohérent depuis Web et Android hors du poste local.
P0	Faire relire l'administration, définir le processus de récupération du compte et documenter l'exploitation des secrets.	Administration maintenable et accès maîtrisés.
P0	Raccorder les annonces et événements à une source/API réelle, puisque cette route n'est pas incluse dans le backend actuel.	Contenus dynamiques fiables sur l'accueil et l'écran événements.
P1	Obtenir les adresses/coordonnées GPS validées des trois églises et un vrai lien de téléchargement public.	Navigation cartographique précise et invitations utiles.
P1	Réaliser et tester le parcours demande de prière; confirmer la source et les droits des contenus bibliques.	Expérience utilisateur complète et contenus conformes.
P1	Intégrer MTN MoMo officiel (sandbox puis production), callbacks/webhooks authentifiés, références fournisseur uniques et rapprochement.	Paiements réels et statut vérifié de bout en bout.
P2	Prévoir sauvegardes chiffrées, supervision, tests de restauration, politique de rétention et migration vers une base de production si nécessaire.	Résilience, traçabilité et capacité à évoluer.
P2	Renforcer les tests automatisés front-end, API, Android, accessibilité et tailles d'écran.	Réduire les régressions et mesurer la qualité.

Critères de préparation à une mise en production

- URLs HTTPS, politique CORS vérifiée, secrets générés et conservés hors du dépôt.
- Essais de connexion administrateur, contrôles de doublons, sauvegarde/restauration et vérification sur un appareil Android réel.
- Test de tous les liens de partage et de localisation; contenu et coordonnées approuvés par l'église.
- Pour tout paiement : paiement réel jamais simulé, confirmation exclusivement après vérification fiable du fournisseur.
- Documentation de confidentialité, support, incidents, mises à jour et responsables opérationnels.

> **État livré :** application Web responsive et projet Android; inscriptions reliées à l'API; espace d'administration pour le suivi des demandes et la consultation des intentions de don; contrôles anti-doublon et tests API. Les paiements réels, la prière interactive, la source d'annonces et l'hébergement de production restent à finaliser.

Références de projet : `src/` (application client), `server/src/` (API et base), `ANDROID.md` et `server/README.md` (instructions techniques).